

UPVS-Engine v4 — Végleges Megvalósítási Terv (Caching és DAG Audit kiegészítéssel)

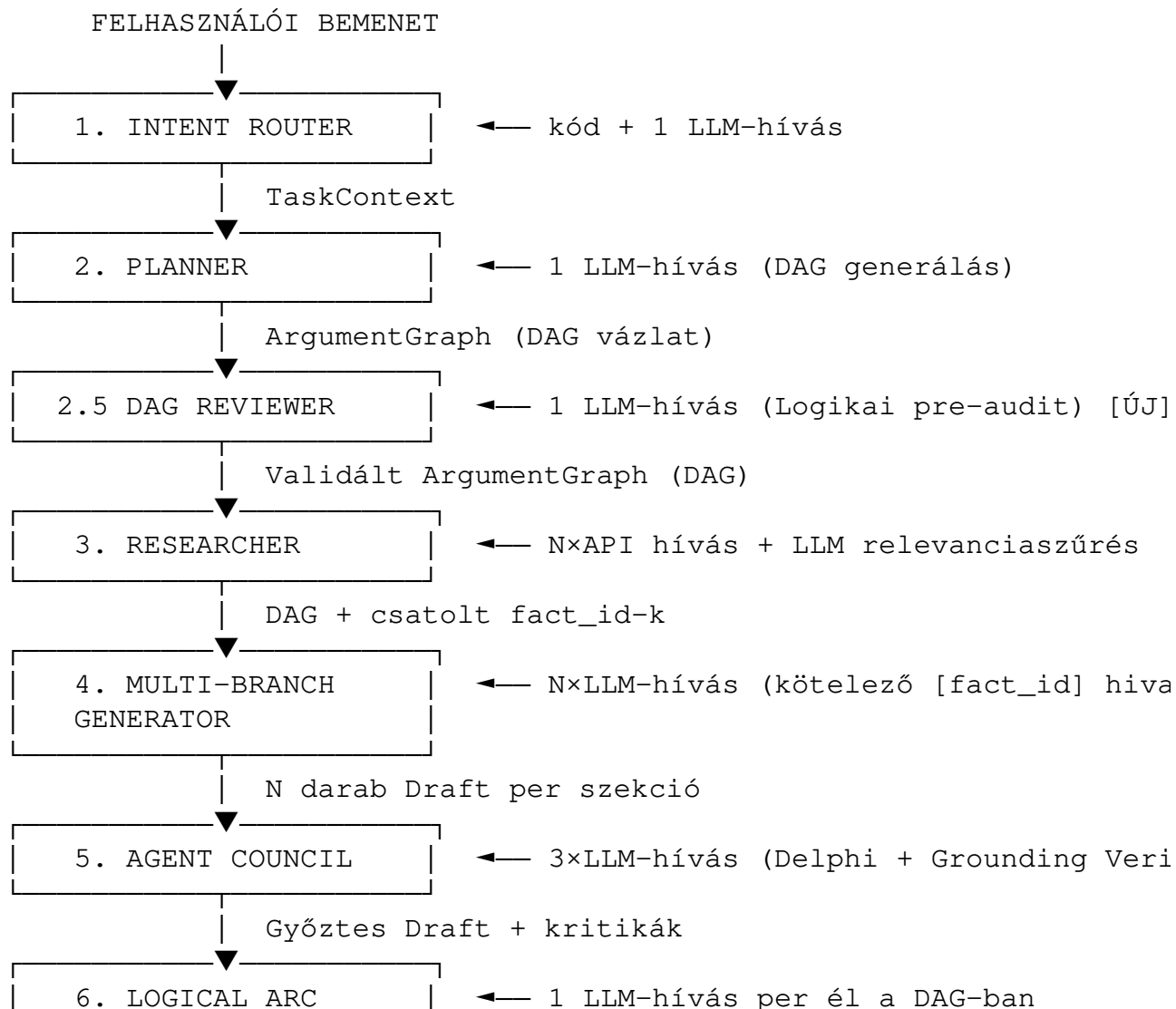
0. Mi ez a rendszer?

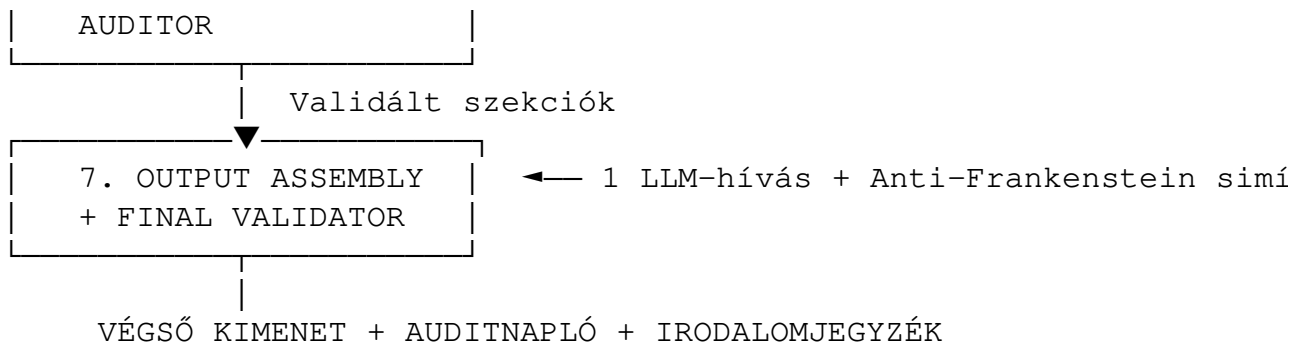
Egy **univerzális prompt-feldolgozó, forráskutató és kimenet-verifikációs motor**, amely bármilyen feladatra képes a felhasználó szándékához **99%-ban illeszkedő**, torzítás-, hallucináció- és tényhibamentes kimenetet előállítani.

A rendszer **nem egy monolitikus AI-ágens**, hanem egy **szoftverarchitektúra**, amelyben:

- A **determinisztikus kód** (Python) vezérli az állapotgépet, a memóriakezelést, a cachinget, a gráfbejárást, a pontozás-aggregációt, a retry-logikát, a forrás-adatbázist és a perzisztenciát.
- Az **AI (LLM-hívások)** kizárólag ott dolgozik, ahol szemantikai megértés szükséges: generálás, kritika, logikai ellenőrzés, forrás-relevanciaszűrés.

1. Architektúra áttekintés





2. Rétegek részletes leírása

2.1. INTENT ROUTER (Szándék-osztályozó)

Meghatározza a kérés kategóriáját, betölti a rubrikát, a Few-Shot példákat (max. 3 db), és **dinamikusán beállítja a kutatás mélységét** (`research_depth` paraméter), hogy elkerüljük az "overkill"-t ott, ahol nem indokolt:

- **None (Kikapcsolva):** Kódolás, fordítás, kreatív írás (0 API hívás).
- **Shallow (Sékély):** Általános blogposzt, lazább elemzés (Webes keresés, max 1 hívás/node).
- **Standard (Normál):** Üzleti döntéstámogatás, cikk (Web + OpenAlex, 2 hívás/node).
- **Deep (Mély):** 99%-os pontosságú tudományos/akadémiai esszé (OpenAlex, PubMed, arXiv, 5 hívás/node, szigorú `[fact_id]` kényszer).

2.2. PLANNER (DAG Váztervező)

(Változatlan a v3-hoz képest) A dokumentum logikai vázát DAG (Directed Acyclic Graph) formában hozza létre, beleértve a csomópontok címeit, állításait és a specifikus kutatási keresőkérdéseket (`research_queries`).

2.5. DAG REVIEWER (Váz-ellenőrző és Logikai Pre-Audit) — ÚJ

Mielőtt elégetnénk a drága API hívásokat és feldolgozási időt a kutatásra (3. réteg), meg kell bizonyosodnunk arról, hogy a gráf logikailag helyes és megállja a helyét.

Mit csinál:

1. **AI (1 hívás):** A Planner által generált DAG-ot egy "Szenior Logikai Építész" prompt vizsgálja meg.
2. **Kérdések, amikre az AI válaszol:**
 - Értelmesek és védhetőek-e az alapfeltevések (axiómák)?
 - A premisszákból tényleg levezethető-e a konklúzió?
 - Vannak-e logikai ugrások a tervezett struktúrában, ami miatt további köztes csomópontokra lenne szükség?
 - A `research_queries` keresőszavak kellően specifikusak-e ahhoz, hogy jó eredményt hozzanak (nem túl tágak)?

3. Kimenet és Kód logika:

- Ha a Reviewer hibát talál: visszadobja a Plannernek (max. 2 iteráció), hogy

javítsa a DAG-ot a kritikák alapján.

- Ha jóváhagyja: a rendszer tovább lép a Researcher rétegre.

2.3. RESEARCHER (Forráskutató & Fact Store)

A jóváhagyott DAG `research_queries` paraméterei alapján párhuzamos API hívásokat indít (OpenAlex, PubMed, arXiv, Web). Az eredményeket LLM-mel szűri, minősíti, majd SQLite `fact_store`-ba menti. Ha nincs forrás, `[no_source]` jelölést kap a csomópont.

Döntési pont: Miért a generálás ELŐTT kutatunk (RAG), és miért nem utólag validálunk?

1. **A Hallucinációs Spirál elkerülése:** Ha az AI "fejből" ír meg egy szöveget, és tele van fals tényekkel, az utólagos tényellenőrzés (fact-checking) és újraíratás gyakran végtelen "írd újra -> megint rossz" ciklushoz vezet.
2. **Kényszerített Lehorgonyzás (Grounding):** Ha előre a generátor kezébe adjuk a tényeket, és kikötjük, hogy *kizárólag* az adatbázisban lévő `[fact_id]`-k felhasználásával érvelhet, drasztikusan csökkentjük a kitalált információk (hallucinációk) esélyét. Ez a 99%-os ténszerűség fundamentuma.

2.4. MULTI-BRANCH GENERATOR (Többágú Generátor)

(Változatlan a v3-hoz képest) Szekciónként 3 draftot generál (Konzervatív, Kritikai, Szintetikus), szigorúan a `fact_store`-ból húzott tényekre építve, kötelező `[fact_id]` hivatkozásokkal.

2.5. AGENT COUNCIL (Delphi Protokoll + Grounding Verifier)

(Változatlan a v3-hoz képest) Négytagú tanács (Domain Expert, De-biaser, Logical Arc Auditor, Grounding Verifier) értékeli és választja ki a legjobb draftot. A De-biaser vagy a Grounding Verifier vétója újragenerálást vált ki.

2.6. LOGICAL ARC AUDITOR (Globális Logikai Ív Ellenőrző)

(Változatlan a v3-hoz képest) Az elfogadott szekciók között ellenőrzi a logikai átmeneteket és kiszűri a belső ellentmondásokat az ellentmondás-mátrix segítségével.

2.7. OUTPUT ASSEMBLY + FINAL VALIDATOR (Anti-Frankenstein Edit) — MÓDOSÍTVA

Amikor tényekből (`[fact_id]`) szigorúan építkezünk, a szöveg hajlamos "darabossá", robotikussá válni (Frankenstein-szöveg). Ennek elkerülésére az Assembly réteg egy speciális Prompting stratégiát kap.

Mit csinál:

1. **Kód:** Topológiai sorrendben összefűzi a validált szekciókat.
2. **AI (1 hívás - Anti-Frankenstein Editor):**
 - **Szabály:** "A szöveg TARTALMÁN és a TÉNYEKEN, valamint a hivatkozások helyén (`[fact_id]`) SZIGORÚAN TILOS változtatni."
 - **Feladat:** Simítsd el az átvezetéseket a bekezdések között. Alakítsd át a lexikonszerű felsorolásokat folyékony, élvezhető, professzionális prózává,

megfelelő diskurzus-jelölők (következésképpen, ezzel szemben, ebből fakadóan) használatával. Hozd egyensúlyba a ritmust.

3. **Kód:** A `[fact_id]` markereket elegáns lábjegyzetekké (`[1]`, `[2]`) alakítja, és legenerálja az Irodalomjegyzéket a `fact_store` alapján.
4. **Kód:** Végző rubrika kiértékelés és elmentés.

3. Memóriakezelés és Caching (Memory Pool) — ÚJ

Egy 99%-os pontosságú, 7 rétegű rendszer rengeteg AI API hívást és külső keresést igényel. Ezt az erőforrást és időt drasztikusan optimalizálni kell.

3.1. Kétrétegű Caching Stratégia (SQLite Cache)

A rendszer egy dedikált `upvs_cache.db` SQLite adatbázist használ két szinten:

A. External API Cache (Kutató API-khoz):

- Ha a Planner ugyanazt a keresőkérdést generálja egy adott témára (pl. *"danish flexicurity employment rate 2023"*), a kód megnézi az SQLite API cache-t.
- Ha az eredmény 7 napnál frissebb, a teljes OpenAlex / PubMed hálózati kérés kihagyásra kerül.
- **Megtakarítás:** Drasztikusan csökkenti a futásidőt és elkerüli a Rate Limit tiltásokat.

B. Semantic LLM Cache (Generáláshoz és Validáláshoz):

- Bizonyos LLM hívások determinisztikusak lehetnek. Pl. az Intent Router kategorizáló hívása, vagy az Ellentmondás-mátrix egy adott "A" és "B" mondatpárosítása.
- Az LLM hívás payload-jának (bemeneti kontextus + hőmérséklet) MD5 hash-ét eltároljuk a válaszzal együtt.
- Ha pontosan ugyanazt kell kérdezni az LLM-től, azonnal a cache-ből adjuk vissza a választ (API költség és idő = 0).

3.2. State Management (Memory Pool)

A UPVS-Engine egy **State Object**-et (`UPVSEngineState`) utaztat végig a memóriában a futás során. Ez a memóriapool biztosítja, hogy ne kelljen mindent folyton beolvasni, de szükség esetén lehessen onnan folytatni, ahol megállt.

```
class UPVSEngineState:
    session_id: str
    task_context: TaskContext          # Router eredménye
    argument_graph: ArgumentGraph      # Jelenlegi DAG állapot
    fact_store_ref: str                # Hivatkozás az SQLite tény-adat
    section_drafts: dict[str, list]    # szekció_id -> generált draftok
    council_scores: dict[str, dict]    # szekció_id -> pontozások
    audit_trail: list[dict]            # Minden döntés és hiba logja

    # Checkpoint mechanizmus
    def save_checkpoint(self): ...
    def load_checkpoint(self): ...
```

- **Helyi memória (RAM):** A `UPVSEngineState` a futás alatt a memóriában (RAM) él, optimalizálva az azonnali hozzáférést a Python számára.
 - **Checkpointing:** A DAG tervezése után, a kutatás után, és a szekciók generálása után a state automatikusan diszkre mentődik (JSON/Pickle fájlként a munka könyvtárban). Ha megszakad a futás, a rendszer másodpercek alatt újraindul onnan, ahol tartott.
-

4. Megvalósítási lépések (Módosítva)

(A korábbi lépések kiegészülnek a Cache/Memory Pool és DAG Reviewer feladatokkal.)

Fázis	Új/Módosított Feladatok (Hozzáadott idő: ~2 óra)
Fázis 1	1.4 Cache Manager és Memory Pool (<code>state_manager.py</code>) kialakítása (45 perc)
Fázis 2	2.3 DAG Reviewer (Logikai pre-audit) LLM hívás + retry logika megírása (45 perc)
Fázis 3	Az API-hívások integrálása a frissen létrehozott SQLite API Cache-sel
Fázis 6	A 6.1 Output Assembly promptjának kiegészítése az Anti-Frankenstein szabályokkal

Összesített becsült fejlesztési idő: ~17 óra.

5. Ellenőrzési terv kiegészítése

Teszt 6: DAG Reviewer Visszadobás

- **Bemenet:** Egy szándékosan illogikus váz, amit a Planner generált (pl. "A fű zöld → Tehát az ég kék").
- **Elvárt:** A DAG Reviewer (2.5 réteg) észreveszi a logikai ugrást, megtagadja a gráf jóváhagyását, és újratervezést kér.

Teszt 7: Caching Hit Rate

- **Bemenet:** Ugyanazon prompt lefuttatása kétszer egymás után.
- **Elvárt:** A második futtatás során az Intent Router, a Researcher keresései és a statikus LLM értékelések a cache-ből térnek vissza. A futásidő az eredeti ~10%-ára csökken.